Persistent Segment Trees

Introduction

Segment Trees are a fairly powerful way to implement range queries for data structures that also need to be updated. Suppose you have an array of data in which you will often want to both update the array and compute the result of a function over arbitrary ranges. With a segment tree we can perform both updates on our data structure and computation of our functions in *O(logn)* time.

This is really good! There are many cases in which we would want this kind of fast functionality. However, there are also many situations in which we would need to query previous versions of our segment tree. Currently, a standard segment tree is incapable of doing this, and the naive approach (new segment tree each update) costs O(n) space per update, so q updates cost O(nq) total, which is wasteful when most of the tree is unchanged between versions.

It seems like a data structure based on segment trees that could query previous versions without expanding space constraints very quickly would be very useful (undo/redo systems, versioned databases, etc.). But how would we actually implement this in practice? Perhaps we could use the fact that each node update only affects a single path through the segment tree, which would allow us to get O(qlogn) time for q queries.

Solution Approach

We will start with a quick briefing on segment trees. The segment tree is able to achieve O(logn) time on its updates and range queries by recursively the initial array breaking down ranges into ranges with half the length. Each node stores the result of the function of interest applied on that particular range. When we update a single element in a segment tree, only the nodes along the path from the root to that leaf actually change. For an array of size n, that path has length O(log n). Every other node in the tree, which is the vast majority of them, remains identical to the previous version. Copying the entire tree on each update wastes space duplicating nodes thatwe do not need to change.

Persistent segment trees exploit this observation by building each new version as an overlay on top of the previous one. Instead of mutating nodes in place or cloning the whole structure, we create new copies only of the nodes that lie along the update path, and we let these new nodes point to the old, unchanged subtrees for everything off the path. Each version of the tree therefore shares the bulk of its structure with earlier versions, and the only freshly allocated memory per update is the O(log n) chain of nodes we actually had to touch.

An update on version v proceeds recursively from the root. At each node on the path toward the target leaf, we allocate a new node. One of its children is a brand new node that we will continue to build recursively, corresponding to the side of the tree containing the updated index. The other child is simply the pointer we inherit from the same node in version v, reused verbatim. When the recursion reaches the leaf, we write the new value into the freshly allocated leaf node and return. As the recursion unwinds, each new internal node recomputes its stored aggregate from its two children in the usual way. The return value of the top-level call is the root of version v+1, which we save in an array of roots indexed by version number.

Queries work exactly like queries on an ordinary segment tree, with one small adjustment: rather than starting at a fixed global root, we start at the root associated with whichever version we want to query. Because every version is a fully formed segment tree in its own right, the traversal logic is unchanged, and a range query still runs in O(log n) time. To answer a query on version k, we look up roots[k] and descend from there.

The cost analysis falls out cleanly. Each update allocates O(log n) new nodes and leaves the older versions untouched, so q updates use O((n + q log n)) total space, with the initial n coming from building version zero. Both updates and queries run in O(log n) time per operation, matching the performance of a standard segment tree while adding full access to history. Compared to the naive O(nq) approach of snapshotting the whole array on every update, this is a dramatic improvement.

Implementation

Concretely, we represent nodes as objects (or indices into arrays) that store a value together with explicit pointers to a left child and a right child, rather than relying on the implicit 2i and 2i+1 indexing scheme of a classical segment tree. The implicit scheme bakes in the

assumption that the tree lives in one contiguous block and has one canonical shape, which is precisely what we need to abandon. With explicit pointers, a node is free to have children that belong to an older version, and nothing in the data structure cares which version a given child came from.

Building the persistent segment tree works the same as building a normal one. Notice how children are stored as pointers in the node structure.

Algorithm 1 Build initial tree over range $[l, r]$

```
function BUILD(l, r)
    node = new Node
    if l == r then
        node.value = arr[l]
    else
        m = (l + r)/2
        node.leftChild = BUILD(l, m)
        node.rightChild = BUILD(m + 1, r)
        node.value = node.leftChild.value + node.rightChild.value
    return node
```

Below is a pseudocode algorithm for update: Notice how in each recursive call we only create a new node once, which means that we only create new nodes on the path down to the value getting updated.

Algorithm 2 Point update: returns root of new version

```
function UPDATE(prevNode, l, r, pos, val)
    newNode = new Node()                                   ▷ only clone nodes on the path
    if l = r then
        newNode.value = val
    else
        m = (l + r)/2
        if pos ≤ m then
            newNode.leftChild = UPDATE(prevNode.leftChild, l, m, pos, val)
            newNode.rightChild = prevNode.rightChild                                ▷ reuse!
        else
            newNode.leftChild = prevNode.leftChild                                  ▷ reuse!
            newNode.rightChild = UPDATE(prevNode.rightChild, m + 1, r, pos, val)
        newNode.value = newNode.leftChild.value + newNode.rightChild.value
    return newNode
```

The pseudocode for query acts the same as for an original segment tree. The key difference is through the fact that the child of a node can point to a node in an earlier version.

Algorithm 3 Range sum on a specific version

```
function QUERY(node, l, r, ql, qr)
    if qr < l or r < ql then return 0
    if ql ≤ l and r ≤ qr then return node.value
    m = (l + r)/2
    return QUERY(node.leftChild, l, m, ql, qr) + QUERY(node.rightChild, m + 1, r, ql, qr)
```

To find the correct node to start the query, we store a list of versions in an auxiliary array. Each index of the array corresponds to a version, and the entry corresponds to where the root node of that version is stored in the node array.

Summary of Programming Challenge

Our programming challenge focuses on making sure that students develop a keen understanding of how persistent segment trees and how they differ from regular segment trees while also getting them to think about how checksums and versioning works in the context of ranges.

The update and rangeXOR methods ensure that students will build the knowledge and understanding of persistent segment tree implementation. They will have to handle the complexities of child pointers and memory allocation for this unique problem. However, this alone would not be a particularly interesting programming assignment, so we wanted to add more. This brings us to the second learning objective of this homework. We hope this homework will teach students the practical use of hashing as a tool for verifying equality of structured data. We hope this combination of ideas forces them to reason carefully about correctness and collision resistance without getting too deep into the weeds of these fairly advanced concepts.

The key conceptual twist of the assignment lies in the requirement to efficiently identify the first index at which two versions of the array differ. Rather than comparing arrays element by element, which would take linear time, students must leverage the segment tree structure and stored checksums to perform this comparison in logarithmic time. By recursively comparing

corresponding nodes in two versions, the algorithm can skip entire segments when their checksums match and only descend into subtrees where differences are detected. This transforms the segment tree into a mechanism for fast “diff” operations between versions, a capability that goes beyond standard implementations and highlights the power of combining hierarchical data structures with hashing techniques.

Key Ideas for Programming Challenge

The main challenge is the diverge function. The naive approach which recurses into both subtrees and compares the leaves is O(n). The better approach is to use the data that each node holds, the xor of its subtree and the hash as an identity of sorts for the entire subtree. If it’s the same node or the hashes and xor match, the subtree is identical and can be skipped. This reduces diverge to only recurse into subtrees that have potential to diverge which is O(logn). This is because at most only one diverging path exists.

This is appropriately difficult because we provide the hash function and the constants to use in order to prevent collisions. The math portion is given because otherwise it would be exponentially difficult to find the proper numbers to avoid collisions. While still challenging students on implementing the core feature of memory efficient updates with new versions by not copying every single node. Also, implementing a different function like xor instead of just the basic sum, but not too difficult.

Our model solution has these complexities:

- build: O(n) time and space to construct because there are n nodes to initialize
- update: O(logn) time and O(logn) new nodes initialized because pointers are used and only necessary nodes created.
- diverge: O(logn) time because at each level it should check and eliminate one of the two child nodes and goes all the way down to a leaf node.
- rangeXOR: O(logn), basic segment tree query.

Conclusion

Persistent segment trees extend the ordinary segment tree structure to support version-based range queries without significant memory overhead. The key insight is that any single update touches only one node per level of the tree, leaving the rest of the structure unchanged. Rather than duplicating the entire tree for each new version, a persistent segment tree creates new nodes only along the specific root-to-leaf path affected by an update, while having those new nodes point to the unchanged subtrees from the previous version.

This shared-node architecture means all versions coexist in a single flat node array, with each node storing child references through either struct fields or array indices. A separate array of root pointers tracks the entry point for each version, allowing any historical state of the tree to be queried in the same time complexity as a standard segment tree query. This implementation allows the persistent segment tree to maintain the same time complexity as the original segment tree while also only growing by O(logn) space with each query.

Our programming challenge aims to get students experience in coding and implementing these persistent segment trees in the context of functions a bit more complex than sum. We hope to get them thinking about how hashing works as well as how the partitioning of data in segment trees can assist with locating particular divergent branches quickly. The added challenge through the diverge operation should hopefully get the students a better understanding of the works of segment trees and also get them interested in checksums and hashes.